

**LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK
(PBO)**



OLEH :

NIA RAMADHANI
(2311531006)

DOSEN PENGUMPU : Nurfiah, S.ST, M.Kom.

**FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS**

PRATIKUM 1

Desain Aplikasi Laundry

A. Tujuan Praktikum

1. Memahami kembali GUI menggunakan JFrame
2. Memahami penggunaan class, object, encapsulation, constructor, dan method pada Pemrograman Berorientasi Objek

B. Pendahuluan

Pemrograman berorientasi objek (OOP) adalah paradigma pemrograman yang berfokus pada penggunaan **class** dan **object** sebagai dasar pengembangan perangkat lunak. Konsep-konsep utama dalam OOP, seperti class, object, method, dan constructor, sangat penting untuk membangun aplikasi yang terstruktur dengan baik dan mudah dikelola.

Class adalah konsep fundamental dalam OOP yang berfungsi sebagai blueprint atau template untuk membuat objek. Sebuah class mendefinisikan sekumpulan karakteristik dan perilaku yang dimiliki oleh objek-objek yang dibuat berdasarkan class tersebut. Di dalam bahasa pemrograman Java, class dapat berisi data member, method, constructor, nested class, dan interface. Data member adalah atribut yang menyimpan informasi tentang objek, sedangkan method adalah fungsi yang menggambarkan perilaku objek. Constructor adalah metode khusus yang digunakan untuk menginisialisasi objek pada saat pembuatan, sementara nested class dan interface digunakan untuk mendefinisikan struktur tambahan yang mendukung fungsionalitas class.

Object merupakan instansi nyata dari sebuah class dan mencerminkan entitas yang ada di dunia nyata. Setiap objek memiliki **state** (atribut), **behavior** (method), dan **identity** yang membedakannya dari objek lainnya. State mencakup data yang disimpan dalam atribut objek, behavior mencakup operasi yang dapat dilakukan pada objek melalui method, dan identity memastikan bahwa setiap objek dapat diidentifikasi secara unik.

Method adalah blok kode yang dirancang untuk melakukan tindakan tertentu dan dapat dipanggil berulang kali sepanjang program. Di Java, terdapat berbagai method bawaan seperti `main()`, `equals()`, dan `toString()` yang menyediakan fungsionalitas dasar untuk manipulasi objek dan interaksi antar objek. Method mempermudah pengorganisasian kode dan meningkatkan keterbacaan serta pemeliharaan program.

Constructor, di sisi lain, adalah metode khusus yang digunakan untuk menginisialisasi objek dari sebuah class. Constructor secara otomatis dipanggil saat objek dibuat, dan bertugas untuk mengatur nilai awal dari atribut-atribut objek. Ini memastikan bahwa objek dimulai dalam kondisi yang valid dan siap digunakan.

Praktikum ini bertujuan untuk menerapkan konsep-konsep OOP dalam konteks pengembangan aplikasi berbasis Java, khususnya aplikasi laundry. Dalam proses ini, dua package utama dibuat: **model** dan **ui**. Package model berisi berbagai class seperti **User**, **Customer**, **Order**, dan **Service** yang mendefinisikan struktur data dan logika bisnis aplikasi. Sementara itu, package ui berisi **JFrame** untuk antarmuka pengguna, termasuk **LoginFrame** dan **MainFrame**, yang menyediakan tampilan grafis dan interaksi pengguna dengan aplikasi.

Dengan memisahkan logika bisnis dari antarmuka pengguna dan mengimplementasikan fitur login, praktikum ini memberikan pemahaman mendalam tentang penerapan OOP dalam pembangunan aplikasi nyata. Penekanan pada penggunaan class, object, method, dan constructor membantu menciptakan aplikasi yang terorganisir dengan baik, mudah dikelola, dan memenuhi kebutuhan fungsional pengguna.

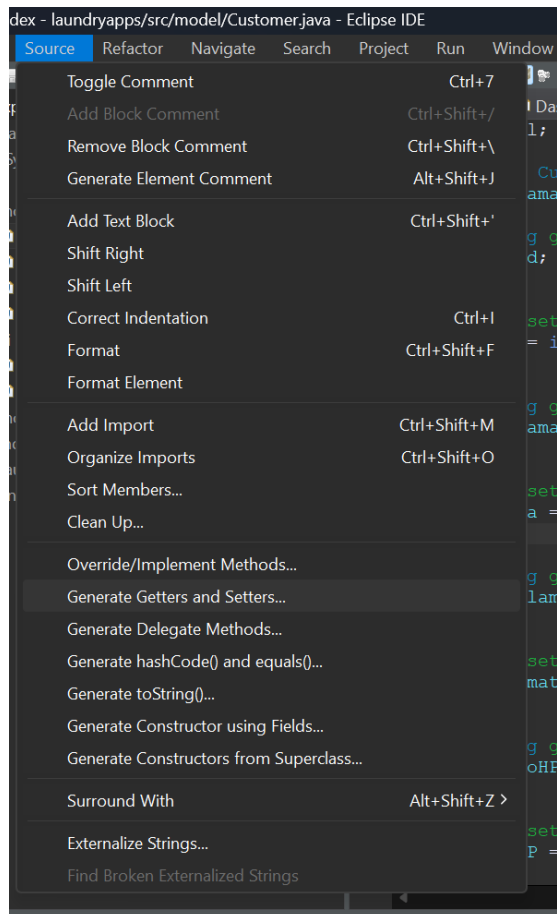
C. Metode Praktikum

1. Buat Java Project baru dengan nama laundryApps, dan buat 2 package untuk model dan ui.
2. Buat class baru pada model dengan nama User dan tambahkan atribut id, nama, username, dan password.

```
package model;

public class User {
    String id, nama, username, password;
}
```

3. Tambahkan getter setter untuk masing-masing atribut. Untuk ini, bisa menggunakan pintasan dengan klik kanan pada laman source > Source > Generate Getters and Setters.



4. Setelah itu, getter dan setter akan ditambahkan secara otomatis.

```
package model;

public class User {
    String id, nama, username, password;

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getNama() {
        return nama;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public String getUsername() {
        return username;
    }
}
```

```

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}

```

5. Tambahkan class lain seperti Customer, Order dan class Service. Masing-masing class diberi atribut dan getter setter nya.

- Class Customer



```

1 package model;
2
3 public class Customer {
4     String id, nama, alamat, noHP;
5
6     public String getId() {
7         return id;
8     }
9
10    public void setId(String id) {
11        this.id = id;
12    }
13
14    public String getNama() {
15        return nama;
16    }
17
18    public void setName(String nama) {
19        this.nama = nama;
20    }
21
22    public String getAlamat() {
23        return alamat;
24    }
25
26    public void setAlamat(String alamat) {
27        this.alamat = alamat;
28    }
29
30    public String getNoHP() {
31        return noHP;
32    }
33
34    public void setNoHP(String noHP) {
35        this.noHP = noHP;
36    }
37 }

```

- Class Order

```

1 package model;
2
3 public class Order {
4     String id, id_customer, id_service, tanggalOrder, tanggalSelesai, status_bayar, status_pesanan, id_user;
5     int total;
6     public String getId() {
7         return id;
8     }
9     public void setId(String id) {
10         this.id = id;
11     }
12     public String getId_customer() {
13         return id_customer;
14     }
15     public void setId_customer(String id_customer) {}
16         this.id_customer = id_customer;
17     }
18     public String getId_service() {
19         return id_service;
20     }
21     public void setId_service(String id_service) {
22         this.id_service = id_service;
23     }
24     public String getTanggalOrder() {
25         return tanggalOrder;
26     }
27     public void setTanggalOrder(String tanggalOrder) {
28         this.tanggalOrder = tanggalOrder;
29     }
30     public String getTanggalSelesai() {
31         return tanggalSelesai;
32     }
33     public void setTanggalSelesai(String tanggalSelesai) {
34         this.tanggalSelesai = tanggalSelesai;
35     }
36     public String getStatus_bayar() {
37
38     }
39     public String getStatus_bayar() {
40         return status_bayar;
41     }
42     public void setStatus_bayar(String status_bayar) {
43         this.status_bayar = status_bayar;
44     }
45     public String getStatus_pesanan() {
46         return status_pesanan;
47     }
48     public void setStatus_pesanan(String status_pesanan) {
49         this.status_pesanan = status_pesanan;
50     }
51     public String getId_user() {
52         return id_user;
53     }
54     public void setId_user(String id_user) {
55         this.id_user = id_user;
56     }
57     public int getTotal() {
58         return total;
59     }
60     public void setTotal(int total) {
61         this.total = total;
62     }
63 }

```

- Class Service

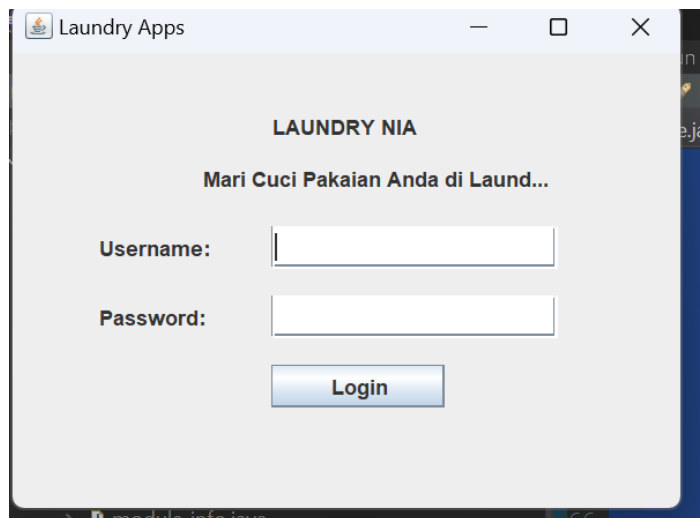
```

1 package model;
2
3 public class Service {
4     String id, jenis, satuan, status;
5     int harga;
6
7     public String getId() {
8         return id;
9     }
10
11    public void setId(String id) {
12        this.id = id;
13    }
14
15    public String getJenis() {
16        return jenis;
17    }
18
19    public void setJenis(String jenis) {
20        this.jenis = jenis;
21    }
22
23    public String getSatuan() {
24        return satuan;
25    }
26
27    public void setSatuan(String satuan) {
28        this.satuan = satuan;
29    }
30
31    public String getStatus() {
32        return status;
33    }
34
35    public void setStatus(String status) {
36        this.status = status;
37    }
38 }

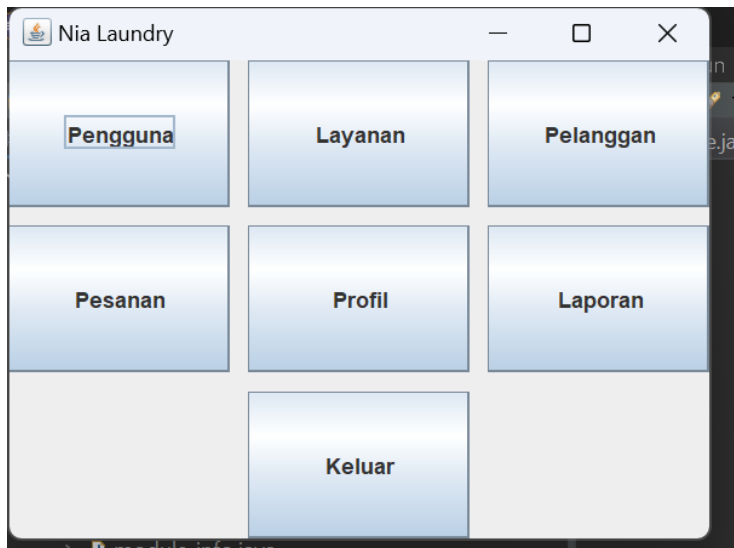
```

6. Selanjutnya, buat GUI menggunakan JFrame. GUI yang dibuat adalah LoginFrame dan MainFrame.

- LoginFrame



- MainFrame



7. Tambahkan method login pada class Users

```
public static boolean login(String username, String password) {
    boolean isLogin = false;
    User user = new User();
    user.setId("1");
    user.setNama("nia");
    user.setUsername("nia");
    user.setPassword("12345");

    if(user.getUsername().equalsIgnoreCase(username.trim())
        && user.getPassword().equals(password)) {
        isLogin = true;
    } else {isLogin=false;}

    return isLogin;
}
```

Method tersebut menggunakan getter setter yang telah dibuat pada class User. equalsIgnoreCase merupakan fungsi untuk mengembalikan String yang menjadi instance tanpa melihat aturan case String tersebut. trim() berfungsi untuk menghapus karakter spasi pada String.

8. Hubungkan LoginFrame dengan class User. Pada source tombol Masuk pada LoginFrame, tambahkan kode berikut:

```
// Memeriksa login menggunakan metode login dari class user
if (user.login(username, password)) {
    // Jika login berhasil, buka MainFrame (buat instance MainFrame jika sudah ada)
    JOptionPane.showMessageDialog(null, "Login Berhasil!");
    // Buat MainFrame atau action lain di sini
    MainFrame mainFrame = new MainFrame(); // Anggap MainFrame sudah dibuat
    mainFrame.setVisible(true); // Tampilkan MainFrame
    frame.dispose(); // Menutup LoginFrame
} else {
    // Jika login gagal
    JOptionPane.showMessageDialog(null, "Gagal Login! Username atau Password salah.");
}
});
}
```

Penjelasan Penghubungan:

- **user.login(username, password):** Method login dari class user dipanggil untuk memverifikasi apakah kombinasi username dan password yang dimasukkan benar.

- Jika metode login mengembalikan true, maka login dianggap berhasil dan akan ditampilkan pesan "Login Berhasil!" serta ditampilkan MainFrame.
- Jika metode login mengembalikan false, akan ditampilkan pesan "Gagal Login! Username atau Password salah."

Method login ini adalah jembatan yang menghubungkan antara input user (username dan password) dari GUI (LoginFrame) dan logika bisnis yang ada di class user untuk verifikasi login.

Jadi, **kode penghubung utama antara LoginFrame dan class user adalah pemanggilan `user.login(username, password)`.**

Method login dari class User digunakan, dengan instance adalah isi dari text field username dan password. `setVisible(true)` untuk menampilkan JFrame MainFrame, dan `dispose()` untuk menutup halaman ini, yaitu LoginFrame.

D. Kesimpulan Praktikum

Praktikum ini menunjukkan penerapan konsep pemrograman berorientasi objek dalam pengembangan aplikasi laundry. Dengan membagi aplikasi menjadi dua package, yaitu **model** dan **ui**, kita memisahkan logika bisnis dari antarmuka pengguna. Pada package model, class-class seperti **User**, **Customer**, **Order**, dan **Service** diciptakan dengan atribut dan getter-setter yang sesuai untuk mendukung fungsionalitas aplikasi. Sementara itu, package ui berisi JFrame untuk tampilan login dan tampilan utama, yang menghubungkan antarmuka pengguna dengan logika verifikasi login.

Metode **login** dalam class **User** digunakan untuk memverifikasi kredensial pengguna dan menghubungkan antarmuka login dengan logika bisnis. Dengan demikian, pengguna dapat login dan mengakses tampilan utama aplikasi setelah berhasil memasukkan username dan password yang benar.

Secara keseluruhan, praktikum ini memberikan pemahaman yang jelas tentang bagaimana class, objek, method, dan constructor bekerja dalam konteks pengembangan aplikasi berbasis GUI di Java, serta bagaimana memisahkan logika aplikasi dari antarmuka pengguna untuk menghasilkan sistem yang lebih terstruktur dan mudah dikelola.

PRATIKUM 2

1.1 Pendahuluan

Pada praktikum ini, saya diharapkan mampu mengimplementasikan fungsi CRUD (Create, Read, Update, Delete) untuk data pengguna menggunakan database MySQL. Praktikum ini bertujuan untuk melatih kemampuan saya dalam membuat tabel pengguna di MySQL, menghubungkan aplikasi Java dengan database, membuat tampilan GUI (Graphical User Interface) untuk CRUD pengguna, serta mengaplikasikan konsep Pemrograman Berorientasi Objek (PBO). Selain itu, saya juga akan mempelajari pembuatan dan penggunaan interface

serta fungsi DAO (Data Access Object) yang dapat memisahkan logika akses data dari logika bisnis, meningkatkan modularitas, dan memungkinkan perubahan dalam pengelolaan data tanpa mempengaruhi bagian lain dari aplikasi. Alat-alat yang dibutuhkan untuk praktikum ini antara lain komputer yang telah terinstal JDK, Eclipse, MySQL atau XAMPP, serta MySQL Connector untuk menghubungkan Java dengan MySQL.

1.2 Tujuan

Tujuan praktikum ini yaitu mahasiswa mampu membuat fungsi CRUD data user menggunakan database MySQL, Adapun poin-poin praktikum yaitu :

- Mahasiswa mampu membuat table user pada database MySQL
- Mahasiswa mampu membuat koneksi Java dengan database MySQL
- Mahasiswa mampu membuat tampilan GUI CRUD user
- Mahasiswa mampu membuat dan mengimplementasikan interface
- Mahasiswa mampu membuat fungsi DAO (Data Access Object) dan mengimplementasikannya.
- Mahasiswa mampu membuat fungsi CRUD dengan menggunakan konsep Pemrograman Berorientasi Objek

1.3 Alat

- Computer / laptop yang telah terinstall JDK dan Eclipse
- MySQL / XAMPP
- MySQL connector atau Connector/J

1.4 Teori

XAMPP yaitu paket software yang terdiri dari Apache HTTP Server, MySQL, PHP dan Perl yang bersifat open source, xampp biasanya digunakan sebagai development environment dalam pengembangan aplikasi berbasis web secara localhost.

Apache berfungsi sebagai web server yang digunakan untuk menjalankan halaman web, MySQL digunakan untuk manajemen basis data dalam melakukan manipulasi data, PHP digunakan sebagai Bahasa pemrograman untuk membuat aplikasi berbasis web.

MySQL adalah sebuah relational database management system (RDBMS) open-source yang digunakan dalam pengelolaan database suatu aplikasi, MySQL ini dapat digunakan untuk menyimpan, mengelola dan mengambil data dalam format table.



Logo MySQL

MySQL Connection/j adalah driver yang digunakan untuk menghubungkan aplikasi berbasis java dengan database MySQL sehingga dapat berinteraksi seperti menyimpan, mengubah, mengambil dan menghapus data. Beberapa fungsi MySQL connector yaitu :

- Membuka koneksi ke database MySQL
- Mengirimkan permintaan SQL ke server MySQL
- Menerima hasil dari permintaan SQL
- Menutup koneksi ke database MySQL

DAO (Data Access Object) merupakan object yang menyediakan abstract interface terhadap beberapa method yang berhubungan dengan database seperti mengambil data (read), menyimpan data(create), menghapus data (delete), mengubah data(update). Tujuan penggunaan DAO yaitu :

- Meningkatkan modularitas yaitu memisahkan logika akses data dengan logika bisnis sehingga memudahkan untuk dikelola
- Meningkatkan reusabilitas yaitu DAO dapat digunakan Kembali
- Perubahan pada logika akses data dapat dilakukan tanpa mempengaruhi logika bisnis.

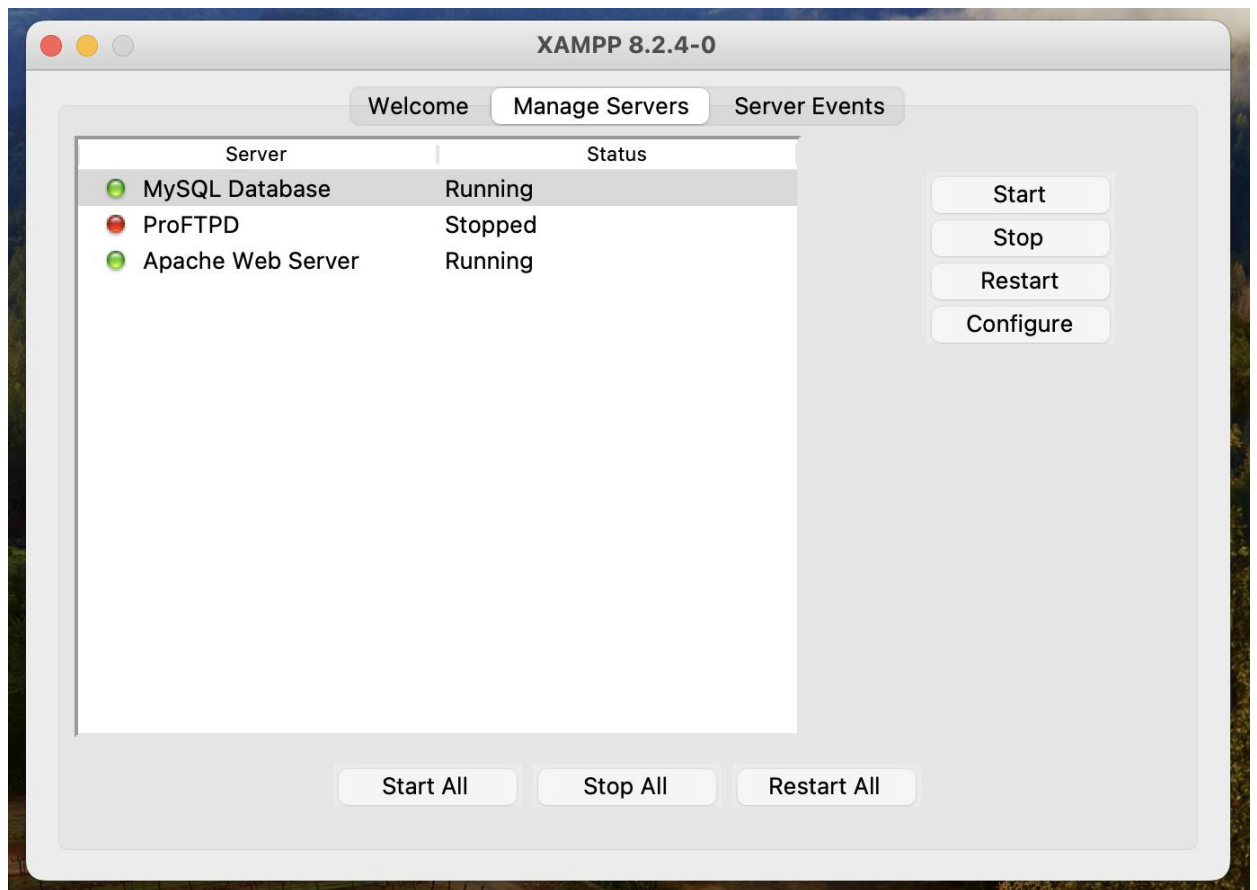
Interface dalam Bahasa java yaitu mendefinisikan beberapa method abstrak yang harus diimplementasikan oleh class yang akan menggunakannya.

CRUD (Create, Read, Update, Delete) merupakan fungsi dasar atau umum yang ada pada sebuah aplikasi yang mana fungsi ini dapat membuat, membaca, mengubah dan menghapus suatu data pada database aplikasi.

1.5 Langkah-langkah

Install XAMPP

- Download XAMPP dari pada link berikut : <https://www.apachefriends.org/>
- Setelah didownload install xampp pada computer/laptop masing-masing
- Jalankan xampp dan aktifkan apache dan mysql



Menambahkan MySQL Connector

Aplikasi java agar dapat terhubung dengan database MySQL membutuhkan sebuah driver yaitu MySQL Connection, berikut ini Langkah-langkah membuat koneksi Database MySQL.

- Download MySQL connection pada link berikut
<https://dev.mysql.com/downloads/connector/j/>

MySQL Community Downloads

Connector/J

General Availability (GA) Releases Archives ⓘ

Connector/J 9.0.0

Select Operating System:

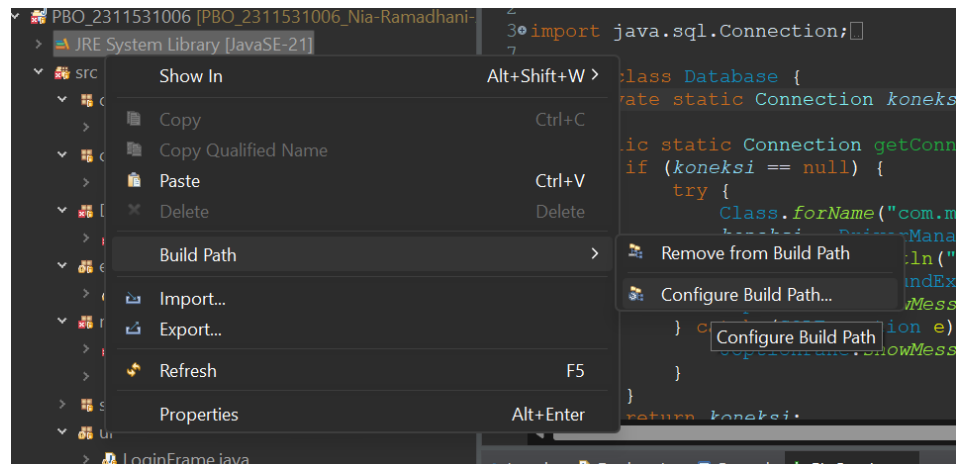
Platform Independent ⓘ

Platform Independent (Architecture Independent), Compressed TAR Archive (mysql-connector-j-9.0.0.tar.gz)	9.0.0	4.3M	Download
		MD5: 820b4d2fa1108130617093a444ee1496 Signature	
Platform Independent (Architecture Independent), ZIP Archive (mysql-connector-j-9.0.0.zip)	9.0.0	5.1M	Download
		MD5: aeaf0db3a50f8756e58eb7a6aa21777d Signature	

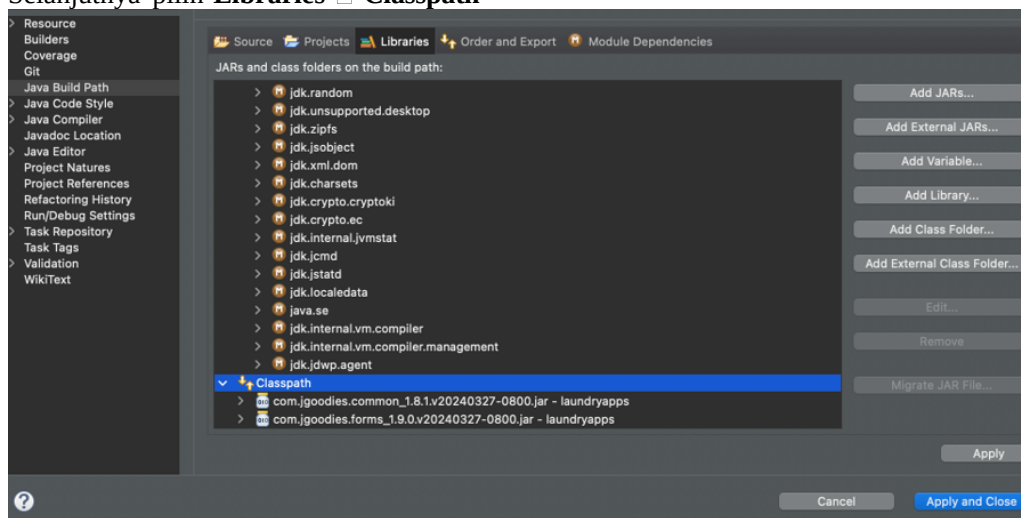
 We suggest that you use the [MD5 checksums](#) and [GnuPG signatures](#) to verify the integrity of the packages you download.

Pilih file yang berekstensi .zip

- Menambahkan MySQL Connector kedalam project dengan cara klik kanan directory **JRE System Library** ☐ **Built Path** ☐ **Configure Build Path**



Selanjutnya pilih **Libraries** ☐ **Classpath**



Tambahkan file MySQL Connector dengan cara klik **Add External JARs**

dan pilih file yang telah didownload dan pilih **Apply and Close**.

Jika berhasil menambahkan MySQL Connector maka akan generate folder Referenced Libraries pada project yang berisi MySQL Connector.



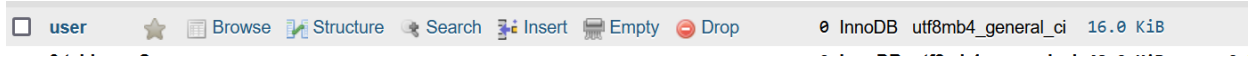
Membuat Database dan Table User

- Buka <http://localhost/phpmyadmin>
- Klik **new** dan **buat** database dengan nama **laundry_apps**

Create database

laundry_apps utf8mb4_general_ci Create

- Buat table user dengan cara klik database laundry_apps dan buat table dengan nama user



Klik create maka akan muncul seperti gambar dibawah ini.

Table structure Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None			Change Drop More
2	name	varchar(128)	utf8mb4_general_ci		No	None			Change Drop More
3	username	varchar(128)	utf8mb4_general_ci		No	None			Change Drop More
4	password	text	utf8mb4_general_ci		No	None			Change Drop More

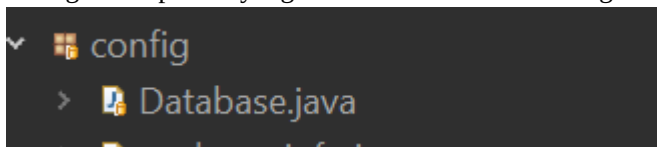
Check all With selected: Review Change Drop Primary Unique Index

Isi seperti gambar diatas dan klik save.

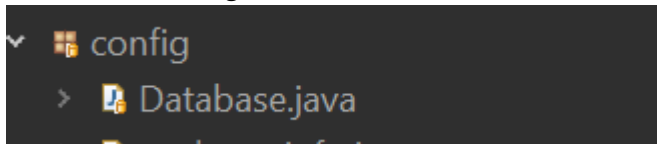
Membuat Koneksi ke Database MySQL

Setelah berhasil menambahkan MySQL Connector maka dapat membuat koneksi ke database MySQL, berikut Langkah-langkahnya.

- Buat package baru dengan nama config, package ini yang akan digunakan untuk membuat konfigurasi aplikasi yang akan dibuat termasuk dengan konfigurasi database.



- Buat class baru dengan nama Database,



kemudian konfigurasi sesuai dengan kode program berikut

```

1 package config;
2
3 import java.sql.Connection;
4
5
6
7
8 public class Database {
9     private static Connection koneksi;
10
11     public static Connection getConnection() {
12         if (koneksi == null) {
13             try {
14                 Class.forName("com.mysql.cj.jdbc.Driver");
15                 koneksi = DriverManager.getConnection("jdbc:mysql://localhost:3306/");
16                 System.out.println("Koneksi berhasil!");
17             } catch (ClassNotFoundException e) {
18                 JOptionPane.showMessageDialog(null, "Driver tidak ditemukan: " + e.getMessage());
19             } catch (SQLException e) {
20                 JOptionPane.showMessageDialog(null, "Koneksi gagal: " + e.getMessage());
21             }
22         }
23         return koneksi;
24     }
25 }

```

Membuat Tampilan CRUD User

- Buat file baru menggunakan JFrame pada package ui dengan nama UserFrame seperti gambar berikut ini.

The image shows a Java Swing window titled "UserFrame". It has a light gray background. There are three text input fields stacked vertically. The first field is labeled "Name", the second "Username", and the third "Password". Below these fields are four buttons arranged horizontally: "Save", "Update", "Delete", and "Cancel". Each button has a blue gradient and a shadow effect.

Keterangan :

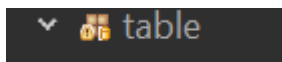
Component	Variable	Keterangan
JTextField	txtName	Name
JTextField	txtUsername	Username

JTextField	txtPassword	Password
JButton	btnSave	Save
JButton	btnUpdate	Update
JButton	btnDelete	Delete
JButton	btnCancel	Cancel
JTable	tableUsers	Table Users

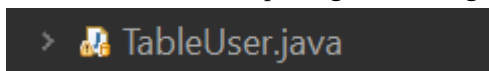
Membuat Table Model

Table model user ini berguna untuk mengambil data dari database dan ditampilkan kedalam table.

- Buat package baru dengan nama **table**



- Buat file baru didalam package table dengan nama **TableUser**,



kemudian isikan dengan kode program berikut.

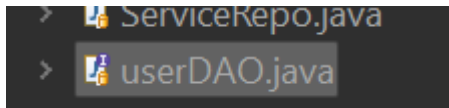
```

1 package table;
2
3 import java.util.List;
4
5 public class TableUser extends AbstractTableModel {
6     private List<User> userList;
7     private String[] columnNames = { "ID", "Name", "Username", "Password" };
8
9     public TableUser(List<User> userList) {
10         this.userList = userList;
11     }
12
13     @Override
14     public int getRowCount() {
15         return userList.size();
16     }
17
18     @Override
19     public int getColumnCount() {
20         return columnNames.length;
21     }
22
23     @Override
24     public Object getValueAt(int rowIndex, int columnIndex) {
25         User user = userList.get(rowIndex);
26         switch (columnIndex) {
27             case 0: return user.getId();
28             case 1: return user.getName();
29             case 2: return user.getUsername();
30             case 3: return user.getPassword();
31             default: return null;
32         }
33     }
34 }

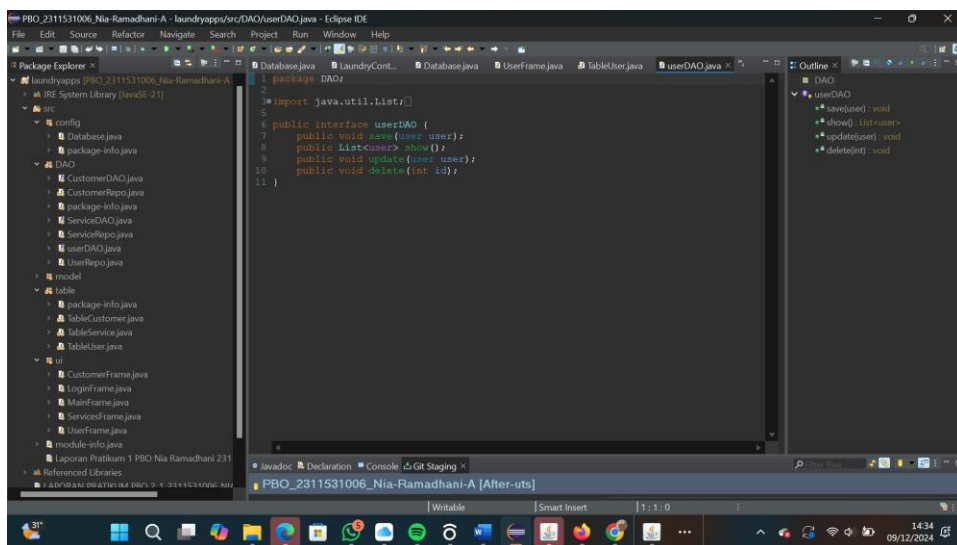
```

Membuat Fungsi DAO

- Buat package baru dengan nama DAO
- Buat class Interface baru dengan nama **UserDAO**,



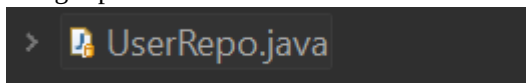
kemudian isikan dengan kode program berikut.



Terdapat method **save, show, delete dan update**. Method pada class interface digunakan sebagai method utama yang wajib diimplementasikan pada class yang menggunakannya.

Menggunakan Fungsi DAO

- Buat class baru pada package DAO dengan nama **UserRepo** yang mana akan digunakan untuk mengimplementasikan DAO yang telah dibuat.



- Implementasikan UserDao dengan kata kunci **implements**



- Masukkan kode di bawah ini

```
1 package DAO;
2
3 import java.util.ArrayList;
4
5
6
7 public class UserRepo {
8     private List<user> userList;
9
10    public UserRepo() {
11        userList = new ArrayList<>();
12    }
13
14    public List<user> show() {
15        return userList;
16    }
17
18    public void save(user user) {
19        user.setId(String.valueOf(userList.size() + 1)); // Simple ID generation
20        userList.add(user);
21    }
22
23    public void update(user user) {
24        for (user u : userList) {
25            if (u.getId().equals(user.getId())) {
26                u.setNama(user.getNama());
27                u.setUsername(user.getUsername());
28                u.setPassword(user.getPassword());
29                break;
30            }
31        }
32    }
33
34    public void delete(String id) {
```

- untuk method **save**, isikan dengan kode program berikut.

```
@Override
public void save(User user) {
    String insert = "INSERT INTO user (name, username, password) VALUES (?, ?, ?)";
    try (PreparedStatement st = connection.prepareStatement(insert)) {
        st.setString(1, user.getNama());
        st.setString(2, user.getUsername());
        st.setString(3, user.getPassword());
        st.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

- Membuat method **show** untuk mengambil data dari database

```

@Override
public List<User> show() {
    List<User> users = new ArrayList<>();
    String select = "SELECT * FROM user";
    try (Statement st = connection.createStatement();
        ResultSet rs = st.executeQuery(select)) {
        while (rs.next()) {
            User user = new User();
            user.setId(rs.getString("id"));
            user.setName(rs.getString("name"));
            user.setUsername(rs.getString("username"));
            user.setPassword(rs.getString("password"));
            users.add(user);
        }
    } catch (SQLException e) {
        Logger.getLogger(UserRepo.class.getName()).log(Level.SEVERE, null, e);
    }
    return users;
}

```

- Membuat method **update** yang digunakan untuk mengubah data.

```

@Override
public void update(User user) {
    String update = "UPDATE user SET name = ?, username = ?, password = ? WHERE id = ?";
    try (PreparedStatement st = connection.prepareStatement(update)) {
        st.setString(1, user.getName());
        st.setString(2, user.getUsername());
        st.setString(3, user.getPassword());
        st.setString(4, user.getId());
        st.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

- Membuat method **delete** yang digunakan untuk menghapus data.

```

@Override
public void delete(String id) {
    String delete = "DELETE FROM user WHERE id = ?";
    try (PreparedStatement st = connection.prepareStatement(delete)) {
        st.setString(1, id);
        st.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

Menggunakan Fungsi CRUD DAO pada GUI

- method digunakan untuk menghapus value inputan Ketika suatu proses berhasil dilakukan, buat method **reset** pada JFrame seperti kode program dibawah ini.

```
public void reset() {  
    txtName.setText("");  
    txtUsername.setText("");  
    txtPassword.setText("");  
    selectedId = null;  
}
```

CREATE USER

- Klik kanan pada tombol **save** ☐ **add event handlers** ☐ **actionPerformed**

kemudian isi dengan kode program berikut.

```
User user = new User();  
user.setNama(txtName.getText());  
user.setUsername(txtUsername.getText());  
user.setPassword(txtPassword.getText());  
usr.save(user);  
reset();  
loadTable();  
JOptionPane.showMessageDialog(null, "User saved successfully!");
```

READ USERS

- Buat method dengan nama **loadTable()** kemudian isikan dengna kode program berikut.

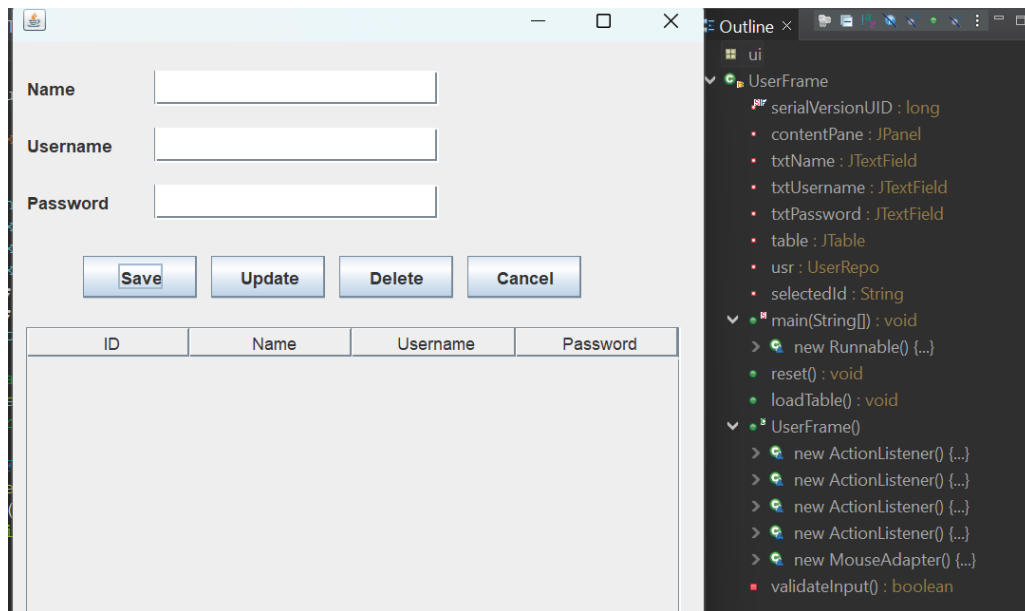
```
public void loadTable() {  
    List<User> ls = usr.show();  
    TableUser model = new TableUser(ls);  
    table.setModel(model);  
}
```

- Memanggil method pada class main, sehingga Ketika pertama kali program dijalankan maka **loadTable** akan dipanggil.

```
UserFrame frame = new UserFrame();  
frame.setVisible(true);
```

UPDATE USER

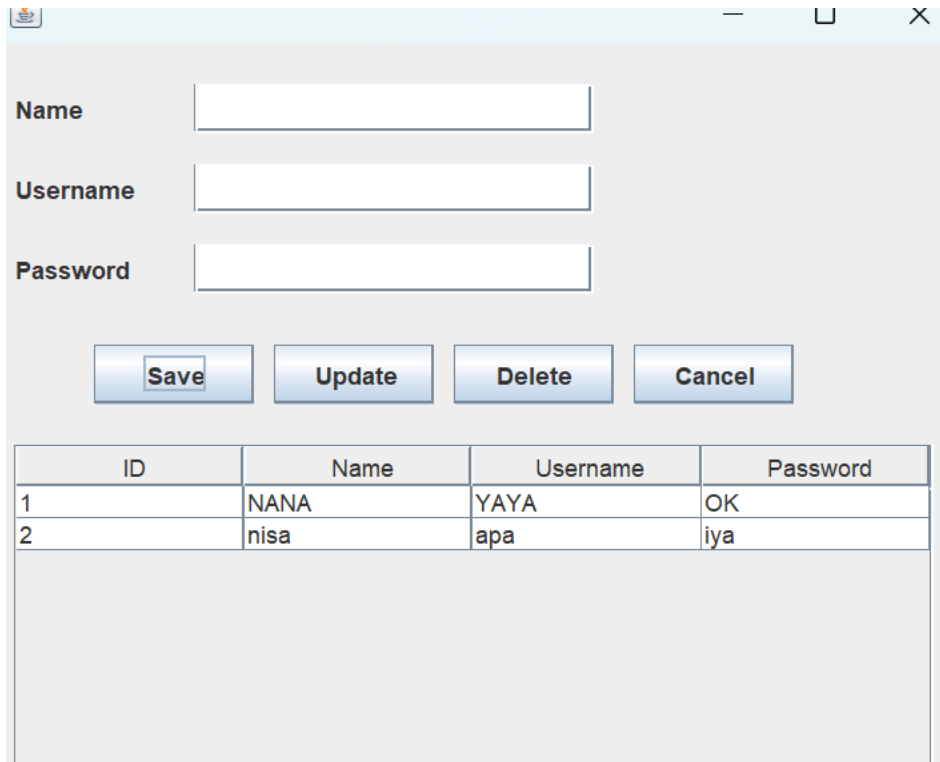
- Klik kanan pada **JTable** ☐ **add event handler** ☐ **mouse** ☐ **mouseClicked**



Isikan dengan kode program dibawah ini.

```
selectedId = table.getModel().getValueAt(row, 0).toString();  
txtName.setText(table.getModel().getValueAt(row, 1).toString());  
txtUsername.setText(table.getModel().getValueAt(row, 2).toString());  
txtPassword.setText(table.getModel().getValueAt(row, 3).toString());
```

- Klik salah satu isi table maka akan secara otomatis tampil pada form inputan.



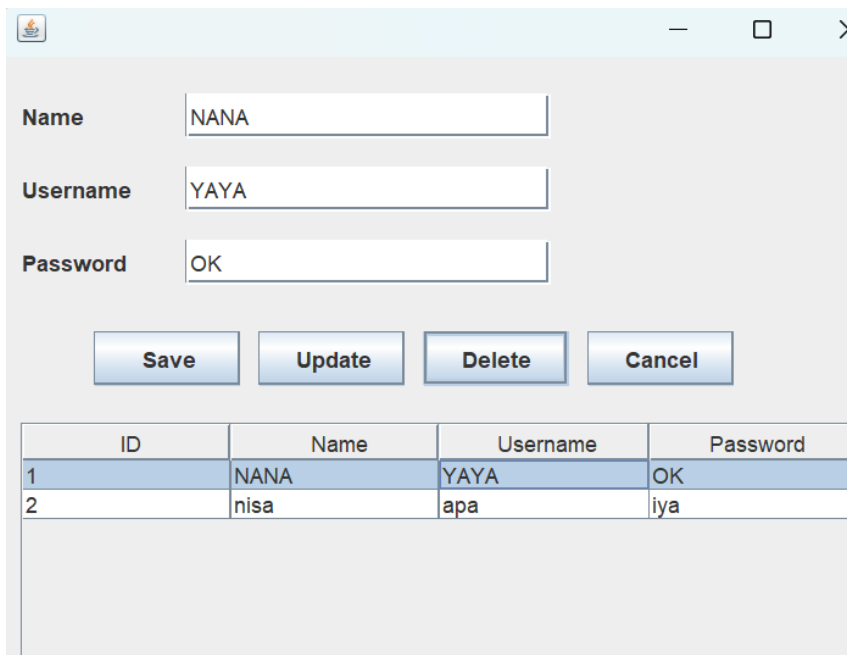
The screenshot shows a Java Swing window with a light blue title bar. Inside, there are three text input fields labeled "Name", "Username", and "Password". Below these fields are four buttons: "Save", "Update", "Delete", and "Cancel". At the bottom of the window is a JTable with the following data:

ID	Name	Username	Password
1	NANA	YAYA	OK
2	nisa	apa	iya

- Klik kanan tombol **update** ☐ **add event handler** ☐ **action** ☐ **actionPerformed** dan isikan dengan kode program berikut.

DELETE USER

- Klik salah satu data pada JTable
- Klik kanan tombol **delete** ☐ **add event handler** ☐ **action** ☐ **actionPerformed**



The screenshot shows the same Java Swing window, but now the form fields are populated with data from the first row of the table: "Name" is "NANA", "Username" is "YAYA", and "Password" is "OK". The "Update" button in the JTable is highlighted, indicating it was clicked.

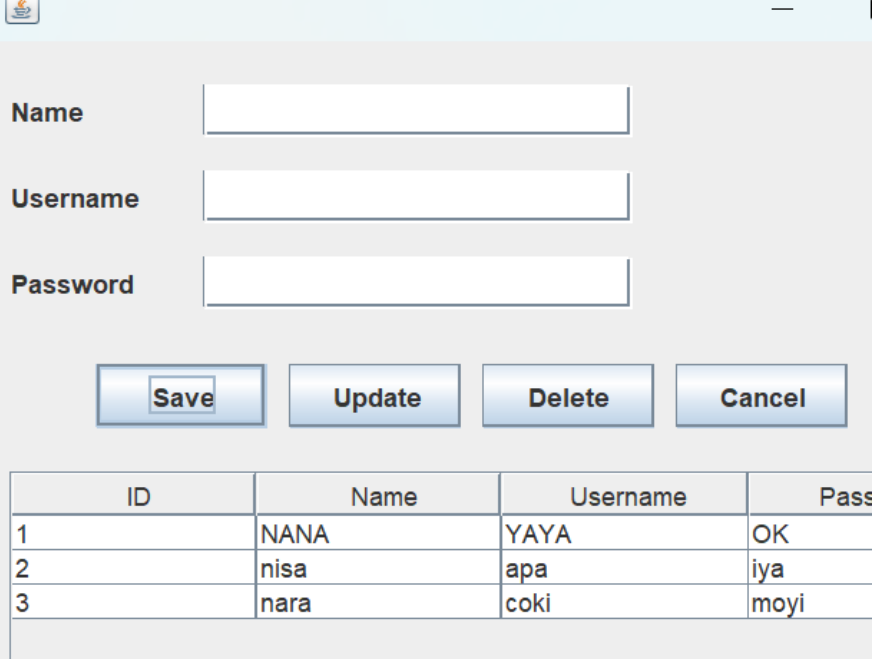
dan isikan dengan kode program berikut.

```

if (selectedId != null && validateInput()) {
    User user = new User();
    user.setId(selectedId);
    user.setName(txtName.getText());
    user.setUsername(txtUsername.getText());
    user.setPassword(txtPassword.getText());
    usr.update(user);
    reset();
    loadTable();
    JOptionPane.showMessageDialog(null, "User updated successfully!");
} else {
    JOptionPane.showMessageDialog(null, "Select a user first!");
}

```

Untuk tombol **save**, digunakan untuk memasukkan data baru

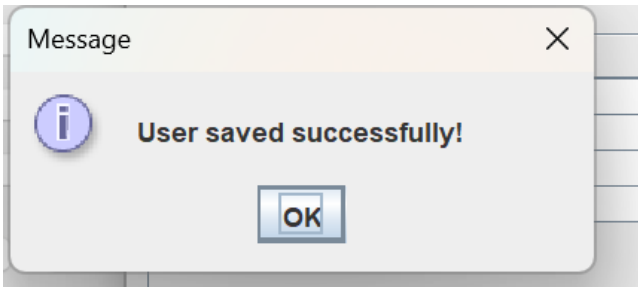


The screenshot shows a Java Swing application window titled "User Management". It contains a form with three text input fields labeled "Name", "Username", and "Password". Below the form are four buttons: "Save", "Update", "Delete", and "Cancel". At the bottom of the window is a table with four columns: "ID", "Name", "Username", and "Password". The table contains three rows of data.

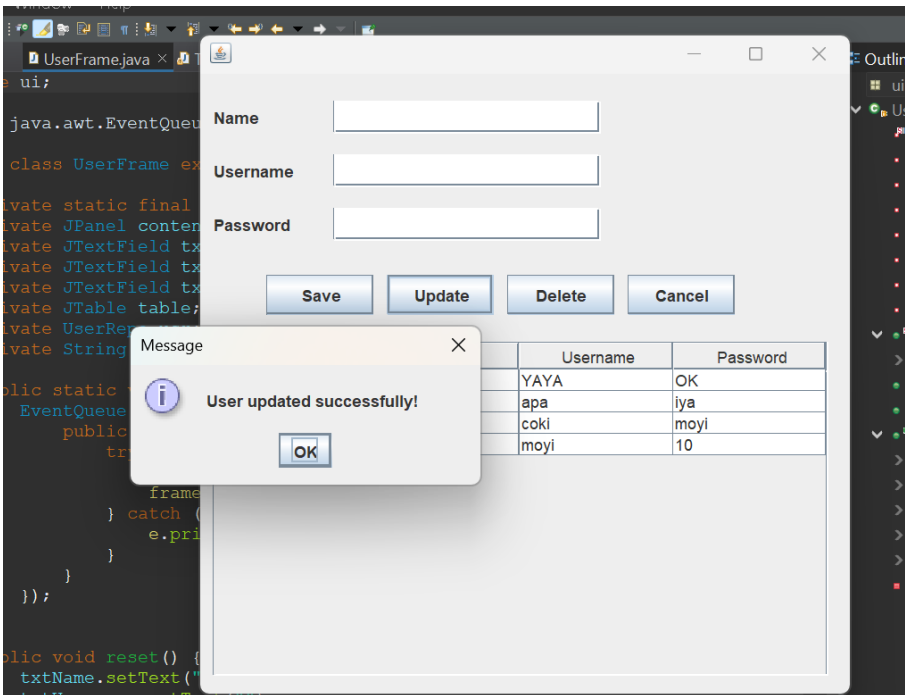
ID	Name	Username	Password
1	NANA	YAYA	OK
2	nisa	apa	iya
3	nara	coki	moyi

Klik

save

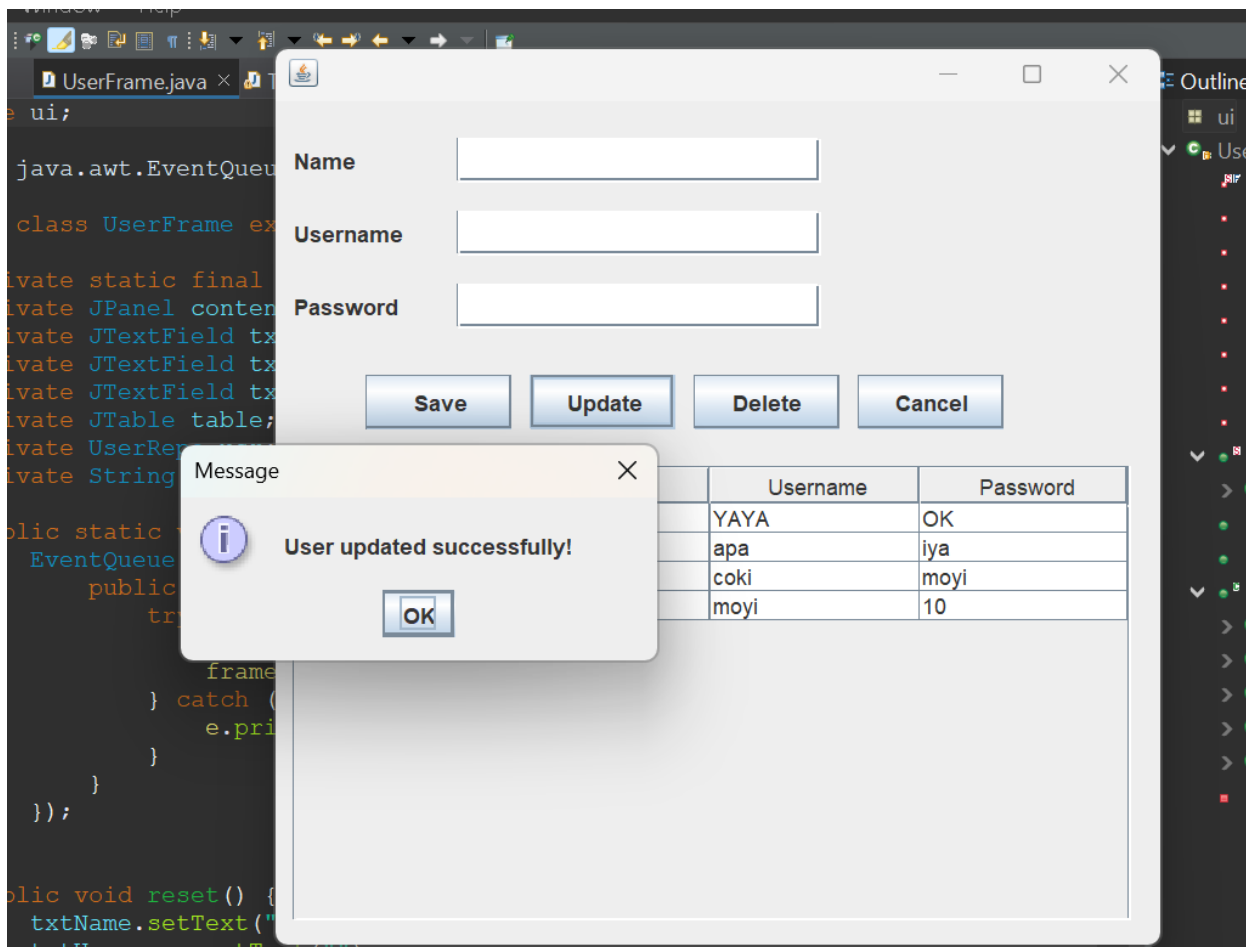


Untuk tombol **update**, digunakan untuk memperbarui data



Klik

update



1.6 Penutup

Melalui praktikum ini, saya telah mempelajari dan mempraktikkan pembuatan fungsi CRUD menggunakan Java dan MySQL dengan menerapkan konsep Pemrograman Berorientasi Objek. Saya juga telah berhasil membuat GUI untuk mengelola data pengguna serta memanfaatkan fungsi DAO untuk modularitas dan pemisahan logika. Dengan pemahaman yang didapatkan, saya diharapkan mampu mengimplementasikan konsep-konsep ini pada proyek-proyek pengembangan perangkat lunak di masa mendatang, terutama yang melibatkan pengelolaan data berbasis database.

PRATIKUM 3

1.1 Pendahuluan

Pada praktikum ini, saya diharapkan mampu mengimplementasikan fungsi CRUD (Create, Read, Update, Delete) untuk data pengguna menggunakan database MySQL. Praktikum ini bertujuan untuk melatih kemampuan saya dalam membuat tabel pengguna di MySQL, menghubungkan aplikasi Java dengan database, membuat tampilan GUI (Graphical User Interface) untuk CRUD pengguna, serta mengaplikasikan konsep Pemrograman Berorientasi Objek (PBO). Selain itu, saya juga akan mempelajari pembuatan dan penggunaan interface serta fungsi DAO (Data Access Object) yang dapat memisahkan logika akses data dari logika bisnis, meningkatkan modularitas, dan memungkinkan perubahan dalam pengelolaan data

tanpa mempengaruhi bagian lain dari aplikasi. Alat-alat yang dibutuhkan untuk praktikum ini antara lain komputer yang telah terinstal JDK, Eclipse, MySQL atau XAMPP, serta MySQL Connector untuk menghubungkan Java dengan MySQL.

1.2 Tujuan

Tujuan praktikum ini yaitu mahasiswa mampu membuat fungsi CRUD data user menggunakan database MySQL, Adapun poin-poin praktikum yaitu :

- Mahasiswa mampu membuat table user pada database MySQL
- Mahasiswa mampu membuat koneksi Java dengan database MySQL
- Mahasiswa mampu membuat tampilan GUI CRUD user
- Mahasiswa mampu membuat dan mengimplementasikan interface
- Mahasiswa mampu membuat fungsi DAO (Data Access Object) dan mengimplementasikannya.
- Mahasiswa mampu membuat fungsi CRUD dengan menggunakan konsep Pemrograman Berorientasi Objek

1.3 Alat

- Computer / laptop yang telah terinstall JDK dan Eclipse
- MySQL / XAMPP
- MySQL connector atau Connector/J

1.4 Teori

XAMPP yaitu paket software yang terdiri dari Apache HTTP Server, MySQL, PHP dan Perl yang bersifat open source, xampp biasanya digunakan sebagai development environment dalam pengembangan aplikasi berbasis web secara localhost.

Apache berfungsi sebagai web server yang digunakan untuk menjalankan halaman web, MySQL digunakan untuk manajemen basis data dalam melakukan manipulasi data, PHP digunakan sebagai Bahasa pemrograman untuk membuat aplikasi berbasis web.

MySQL adalah sebuah relational database management system (RDBMS) open-source yang digunakan dalam pengelolaan database suatu aplikasi, MySQL ini dapat digunakan untuk menyimpan, mengelola dan mengambil data dalam format table.



Logo MySQL

MySQL Connection/j adalah driver yang digunakan untuk menghubungkan aplikasi berbasis java dengan database MySQL sehingga dapat berinteraksi seperti menyimpan, mengubah, mengambil dan menghapus data. Beberapa fungsi MySQL connector yaitu :

- Membuka koneksi ke database MySQL
- Mengirimkan permintaan SQL ke server MySQL
- Menerima hasil dari permintaan SQL
- Menutup koneksi ke database MySQL

DAO (Data Access Object) merupakan object yang menyediakan abstract interface terhadap beberapa method yang berhubungan dengan database seperti mengambil data (read), menyimpan data(create), menghapus data (delete), mengubah data(update). Tujuan penggunaan DAO yaitu :

- Meningkatkan modularitas yaitu memisahkan logika akses data dengan logika bisnis sehinggamemudahkan untuk dikelola
- Meningkatkan reusabilitas yaitu DAO dapat digunakan Kembali
- Perubahan pada logika akses data dapat dilakukan tanpa mempengaruhi logika bisnis.

Interface dalam Bahasa java yaitu mendefinisikan beberapa method abstrak yang harus diimplementasikanoleh class yang akan menggunakannya.

CRUD (Create, Read, Update, Delete) merupakan fungsi dasar atau umum yang ada pada sebuah aplikasi yang mana fungsi ini dapat membuat, membaca, mengubah dan menghapus suatu data pada databaseaplikasi.

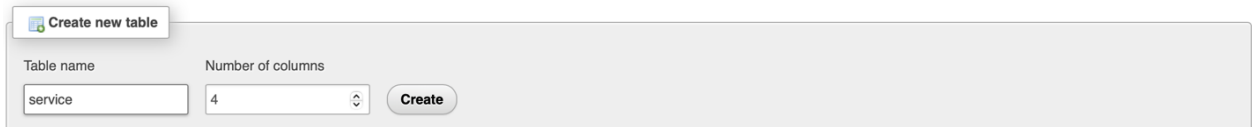
1.5 Langkah-langkah

XAMPP

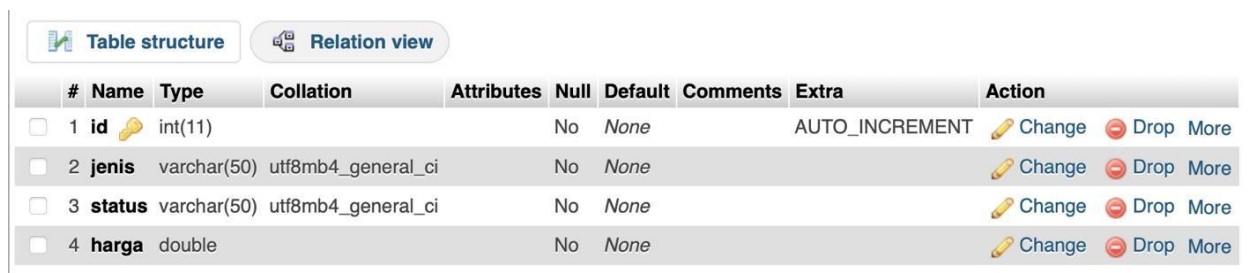
- Jalankan xampp dan aktifkan apache dan mysql

Membuat Database dan Table Service dan Table Costumer

- Buka <http://localhost/phpmyadmin>
- Buat table service dengan cara klik database laundry_apps dan buat table dengan nama service



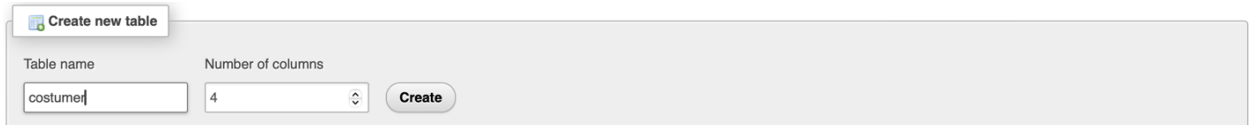
Klik create, kemudian isi tabel dengan 'id', 'jenis', 'status', dan 'harga'. Maka akan muncul



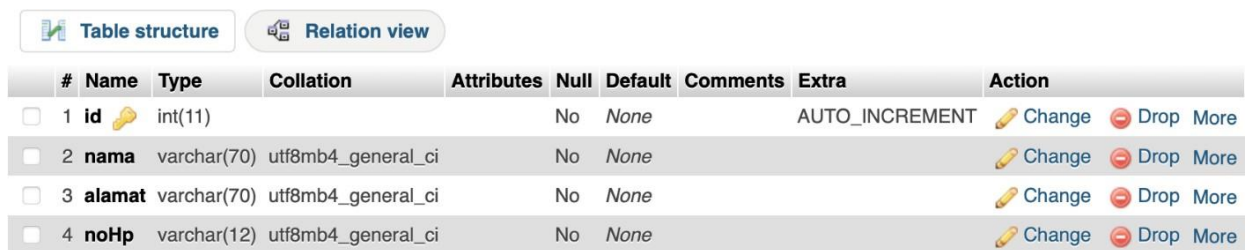
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 jenis	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 status	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 harga	double			No	None			Change Drop More

sepertigambar dibawah ini.

- Buat table service dengan cara klik database laundry_apps dan buat table dengan nama Costumer



Klik create, kemudian isi tabel dengan 'id', 'jenis', 'status', dan 'harga'. Maka akan muncul



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 nama	varchar(70)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 alamat	varchar(70)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 noHp	varchar(12)	utf8mb4_general_ci		No	None			Change Drop More

sepertigambar dibawah ini.

Membuat Tampilan CRUD Service dan Customer

- Buat file baru menggunakan JFrame pada package ui dengan nama ServiceFrame sepertigambar berikut ini.

Nama
Alamat
NoHP

ID	Name	Alamat	NoHP
1	NANA	jln gaja mada	0927383

Keterangan :

Component	Variable	Keterangan
JComboBox	comboJenis	Jenis
JTextField	txtQuantity	Nilai Satuan
JComboBox	comboSatuan	Satuan
JComboBox	comboStatus	Status

JTextField	txtHarga	Harga
JButton	btnSave	Save
JButton	btnUpdate	Update
JButton	btnDelete	Delete
JButton	btnCancel	Cancel
JTable	tableService	Table Service

- Buat file baru menggunakan JFrame pada package ui dengan nama CostumerFrameseperti gambar berikut ini.

The image shows a Java Swing window titled "CostumerFrame". Inside the window, there is a form with three input fields labeled "Nama", "Alamat", and "No HP". Below these fields are four buttons: "Save", "Update", "Delete", and "Cancel". At the bottom of the window, there is a table with a black header row and an empty body. The table has four columns.

Keterangan :

Component	Variable	Keterangan
JTextField	txtNama	Nama
JTextField	txtAlamat	Alamat

TextField	txtNoHp	Nomor HP
JBUTTON	btnSave	Save
JBUTTON	btnUpdate	Update
JBUTTON	btnDelete	Delete
JBUTTON	btnCancel	Cancel
JTable	tableService	Table Service

Membuat Table Model

Table model user ini berguna untuk mengambil data dari database dan ditampilkan kedalam table.

- Buat file baru didalam package table dengan nama **TableService**

> 📄 TableService.java

- Buat file baru didalam package table dengan nama **TableCustomer**

> 📄 TableCustomer.java

kemudian isikan TableService dengan kode program berikut.

```
1 package table;
2
3 import java.util.List;
4
5
6 public class TableService extends AbstractTableModel {
7     private List<Service> serviceList;
8     private String[] columnNames = { "ID", "Jenis", "Satuan", "Status", "Harga" };
9
10    public TableService(List<Service> serviceList) {
11        this.serviceList = serviceList;
12    }
13
14    @Override
15    public int getRowCount() {
16        return serviceList.size();
17    }
18
19    @Override
20    public int getColumnCount() {
21        return columnNames.length;
22    }
23
24    @Override
25    public Object getValueAt(int rowIndex, int columnIndex) {
26        Service service = serviceList.get(rowIndex);
27        switch (columnIndex) {
28            case 0: return service.getId();
29            case 1: return service.getJenis();
30            case 2: return service.getSatuan();
31            case 3: return service.getStatus();
32            case 4: return service.getHarga();
33        }
34    }
35 }
```

kemudian isikan TableCustomer dengan kode program berikut.

```

1 package table;
2
3 import java.util.List;
4
5
6
7 public class TableCustomer extends AbstractTableModel {
8     private List<Customer> customerList;
9     private String[] columnNames = { "ID", "Name", "Alamat", "NoHP" };
10
11
12     public TableCustomer(List<Customer> customerList) {
13         this.customerList = customerList;
14     }
15
16     @Override
17     public int getRowCount() {
18         return customerList.size();
19     }
20
21     @Override
22     public int getColumnCount() {
23         return columnNames.length;
24     }
25
26     @Override
27     public Object getValueAt(int rowIndex, int columnIndex) {
28         Customer customer = customerList.get(rowIndex);
29         switch (columnIndex) {
30             case 0: return customer.getId();
31             case 1: return customer.getNama();
32             case 2: return customer.getAlamat();
33             case 3: return customer.getNoHP();
34             default: return null;

```

Membuat Fungsi DAO

- Pada package DAO



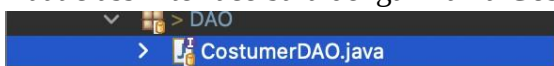
- Buat class Interface baru dengan nama **ServiceDAO**,



kemudian isikan dengan kode program berikut.

```
1 package DAO;
2
3 import java.util.List;
4
5
6
7 public interface ServiceDAO {
8     void save(Service service);
9     List<Service> show();
10    void update(Service service);
11    void delete(int id);
12    void delete(String id);
13
14 }
```

- Buat class Interface baru dengan nama **CostumerDAO**,



kemudian isikan dengan kode program berikut.

```
1 package DAO;
2
3 import java.util.List;
4
5
6
7
8 public interface CustomerDAO {
9     void save(Customer customer);
10    List<Customer> show();
11    void update(Customer customer);
12    void delete(int id);
13    void delete(String id);
14
15 }
```

Terdapat method **save**, **show**, **delete** dan **update**. Method pada class interface digunakan sebagai method utama yang wajib diimplementasikan pada class yang menggunakannya.

Menggunakan Fungsi DAO

- Buat class baru pada package DAO dengan nama **ServiceRepo** yang mana akan digunakan untuk mengimplementasikan DAO yang telah dibuat.

```
> ServiceRepo.java
```

- Buat class baru pada package DAO dengan nama **CostumerRepo** yang mana akan digunakan untuk mengimplementasikan DAO yang telah dibuat.

```
> CustomerRepo.java
```

- Implementasikan ServiceDao dengan kata kunci **implements**

```
12 public class ServiceRepo implements ServiceDAO {
```

- Masukkan kode di bawah ini

```
1 package DAO;
2
3 import java.util.List;
4
5
6
7 public interface ServiceDAO {
8     void save(Service service);
9     List<Service> show();
10    void update(Service service);
11    void delete(int id);
12    void delete(String id);
13
14 }
```

- Implementasikan CostumerDao dengan kata kunci **implements**

```
16 public class CostumerRepo implements CostumerDAO {
```

- Masukkan kode di bawah ini

```

1 package DAO;
2
3 import java.util.List;
4
5
6
7
8 public interface CustomerDAO {
9     void save(Customer customer);
10    List<Customer> show();
11    void update(Customer customer);
12    void delete(int id);
13    void delete(String id);
14
15 }

```

Service

Jenis	nia
Satuan	3000
Status	nyuci
Harga	rp.10.000

ID	Jenis	Satuan	Status	Harga

Pada service, terdapat form **Jenis**, Quantity, Status, Harga. Untuk Jenis digunakan comboBox dengan nisijenis-jenis pakaian, yaitu “pakaian, Seprei, Handuk, Jas, Gaun” .

Selanjutnya adalah **Quantity** untuk nilai satuan dan satuan nya, di mana jika Pakaian maka otomatis terisi Kg dan selain itu dengan satuan Helai.

Kemudian **Status** untuk mengetahui status laundry, menggunakan comboBox, dengan nisi “Dalam antrian, Sedang di cuci, Selesai”

Terakhir **Harga** yang otomatis terisi sesuai dengan harga x satuan. Dimana masing-masing harga yang ditetapkan adalah

```
private final double PAKAIAN_PRICE_PER_KG = 5000;
private final double SEPREI_PRICE_PER_HELAI = 35000;
private final double JAS_PRICE_PER_HELAI = 20000;
private final double GAUN_PRICE_PER_HELAI = 50000;
private final double HANDUK_PRICE_PER_HELAI = 10000;
```

dan untuk 'x quantity' menggunakan

```
case "Pakaian":
    if (satuan.equals("Kg")) {
        harga = quantity * PAKAIAN_PRICE_PER_KG;
    }
    break; case
"Seprei":
    if (satuan.equals("Helai")) {
harga = quantity * SEPREI_PRICE_PER_HELAI;
    }
    break;
case "Handuk":
    if (satuan.equals("Helai")) {
harga = quantity * HANDUK_PRICE_PER_HELAI;
    }
    break; case
"Jas":
    if (satuan.equals("Helai")) {
        harga = quantity * JAS_PRICE_PER_HELAI;
    }
    break; case
"Gaun":
    if (satuan.equals("Helai")) {
        harga = quantity * GAUN_PRICE_PER_HELAI;
    }
    break;
default:
    break;
}
```

Tombol **Save** pada service

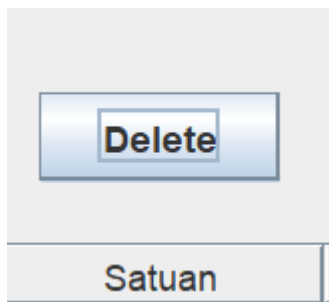
Berhasil menyimpan data yang

telah di masukkan Tombol

Update

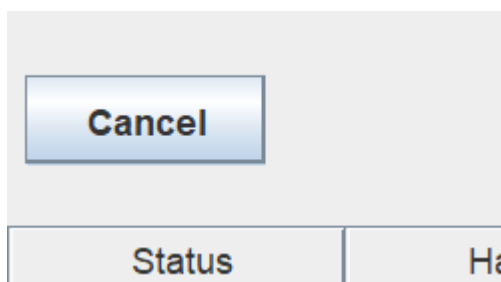
Berhasil merubah data yang telah di masukkan, semula Handuk menjadi Pakaian di nomor 10

tombol **Delete**



Berhasil

menghapu



s data

Tombol

cancel

Untuk men cancel isi form yang telah di isi sebelumnya

1.1 Penutup

Melalui praktikum ini, saya telah mempelajari dan mempraktikkan pembuatan fungsi CRUD menggunakan Java dan MySQL dengan menerapkan konsep Pemrograman Berorientasi Objek. Saya juga telah berhasil membuat GUI untuk mengelola data penggunaserta memanfaatkan fungsi DAO untuk modularitas dan pemisahan logika. Dengan pemahaman yang didapatkan, saya diharapkan mampu mengimplementasikan konsep-konsep ini pada proyek-proyek pengembangan perangkat lunak di masa mendatang, terutama yang melibatkan

pengelolaan data berbasis database.

PRATIUM TERAKHIR

JFRAME OrderDetailFrame

1. Jalankan control panel XAMPP MySql
2. Buat tabel Order Detail dengan SQL sebagai berikut :

The screenshot shows a Java Swing window titled "Laundry Management System". The window is divided into two main sections. On the left is a form for entering order details, and on the right is a table displaying a list of laundry services.

Order Detail Form:

- Order ID:
- Pelanggan:
- Tanggal:
- Tanggal Pengambilan:
- Status:
- Total: Rp10.000
- Pembayaran:
- Status Pembayaran:
-

Table of Laundry Services:

ID	Layanan	Status	Harga	Jumlah	Total
17	Sprei	Sedang Diproses	105000	3	350000
18	Sprei	Dalam Antrian	70000	2	140000
19	Pakaian	Dalam Antrian	25000	5	125000
20	Gaun	Dalam Antrian	50000	1	50000
21	Jas	Dalam Antrian	80000	4	320000

Bottom Section:

Harga/Satuan: Jumlah: Total:

3. Buat desain OrderDetailFrame

Laundry Management System

Order ID:

Pelanggan:

Tanggal:

Tanggal Pengambilan:

Status:

Total: Rp10.000

Pembayaran:

Status Pembayaran:

ID	Layanan	Status	Harga	Jumlah	Total
17	Sprei	Sedang Diproses	105000	3	350000
18	Sprei	Dalam Antrian	70000	2	140000
19	Pakaian	Dalam Antrian	25000	5	125000
20	Gaun	Dalam Antrian	50000	1	50000
21	Jas	Dalam Antrian	80000	4	320000

Harga/Satuan: Jumlah: Total:

4. Panggil tabel dari ServiceFrame ke OrderDetailFrame

```

270 public void loadTable () {
271     List<Service> ls = srv.show();
272     TableService tu = new TableService(ls);
273     tableServices.setModel(tu);
274 }

```

5. Set TableServices pada ScrollPane, jika di klik maka akan otomatis mengisi kolom-kolom kosong yang terdapat di OrderDetailFrame

Yang mana

txtOrderId = Id

cbStatus = Status

lblNilaiTotal & txtTotalOrder = Harga

Jumlah = Quantity

txtHarga = Harga per Pcs

yang dimana berarti yang akan kita inputkan adalah pelanggan, tanggal, tanggal pengambilan, pembayaran, dan status pembayaran yang nantinya akan di simpan dan tampil pada tableDetail

untuk memunculkan saat di klik, maka code addMouseListener di bawah untuk menjalankannya.

```

237 tableServices.addMouseListener(new MouseInputAdapter() {
238     @Override
239     public void mouseClicked(MouseEvent e) {
240         int selectedRow = tableServices.getSelectedRow();
241         if (selectedRow != -1) {
242             // Assuming columns: Order ID, Service Name, Status, Price, Quantity
243             String orderId = tableServices.getValueAt(selectedRow, 0).toString(); // Order ID
244             String serviceName = tableServices.getValueAt(selectedRow, 1).toString(); // Service Name
245             String status = tableServices.getValueAt(selectedRow, 2).toString(); // Status
246             String price = tableServices.getValueAt(selectedRow, 3).toString(); // Price per unit
247             String quantity = tableServices.getValueAt(selectedRow, 4).toString(); // Quantity
248             String harga_pcs = tableServices.getValueAt(selectedRow, 5).toString(); // Quantity
249
250
251             // Populate the fields
252             txtOrderId.setText(orderId);
253             txtTotalOrder.setText(price);
254             lblNilaiTotal.setText(price); // Assuming lblNilaiTotal is a JLabel
255
256             // Set Harga per unit and Jumlah
257             txtHarga.setText(harga_pcs);
258             txtJumlah.setText(quantity);
259
260             // Set the status in the cbStatus combo box based on the status value
261             cbStatus.setSelectedItem(status);
262         }
263     }
264 }
265

```

```

case 0: return service.getId();
case 1: return service.getJenis();
case 2: return service.getStatus();
case 3: return service.getHarga();
case 4: return service.getQuantity(); // Quantity
case 5: return service.getHargaPcs(); // Harga per unit
default: return null;

```

6. Maka jika kita klik salah satu dari tabel, maka hasilnya adalah

ID	Layanan	Status	Harga	Jumlah	Total
17	Sprei	Sedang Diproses	105000	3	350000
18	Sprei	Dalam Antrian	70000	2	140000
19	Pakaian	Dalam Antrian	25000	5	125000
20	Gaun	Dalam Antrian	50000	1	50000
21	Jas	Dalam Antrian	80000	4	320000

Dimana order id, harga/satuan, jumlah, total, status akan terisi secara otomatis sesuai dengan tabel yang di klik.

20	Gaun	Dalam Antrian	50000	1	50000
21	Jas	Dalam Antrian	80000	4	320000

Dimana order id, harga/satuan, jumlah, total, status akan terisi secara otomatis sesuai dengan table.